

# SIMATS ENGINEERING ASSESSMENT TOOL 1

**COURSE NAME: Ethical Hacking**

**COURSE CODE: ITA1407**

---

## COURSE OUTCOME COVERED

**CO4:** *Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.*

---

**QUESTIONS: 01**

**Total Marks:100**

### Annexure A SECURITY TESTING SIMULATION TASK

---

#### ***Code of Conduct:***

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

| S.No. | Assessment Criterion                                 | Excellent                                                                                                                                                           | Good                                                                                     | Satisfactory                                                           | Needs Improvement                                                                          | Marks |
|-------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------|
| 1     | Tool Selection and Configuration(20)                 | Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.                                                    | Selects suitable tools and configures them with minor errors or limited explanation.     | Uses basic tools with partial configuration and limited understanding. | Selects inappropriate tools or fails to configure them correctly.                          |       |
| 2     | Testing Procedure and Execution(20)                  | Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.                                                            | Performs the major testing steps correctly with minor omissions.                         | Completes only basic testing steps with limited documentation.         | Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.                 |       |
| 3     | Identification and Validation of Vulnerabilities(20) | Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.                                        | Identifies and validates most important findings with minor gaps.                        | Identifies basic vulnerabilities but provides limited validation.      | Findings are inaccurate, unsupported, or based only on automated tool output.              |       |
| 4     | Risk Analysis and Remediation(20)                    | Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.                                              | Provides suitable risk analysis and recommendations with minor omissions.                | Provides general risk descriptions and basic remediation measures.     | Risk ratings are unsupported or recommendations are impractical or unrelated.              |       |
| 5     | Evidence, Report Quality, and Ethical Compliance(20) | Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct. | Provides sufficient evidence and a well-organized report with minor presentation issues. | Provides limited evidence or an adequately organized report.           | Evidence is missing, the report is unclear, or ethical and legal requirements are ignored. |       |
|       |                                                      |                                                                                                                                                                     |                                                                                          |                                                                        | Total                                                                                      |       |

## Annexure A

### NOTE:

- Any testing performed outside the faculty-approved laboratory target, authorized scope, or permitted time must be treated as a serious violation. Destructive testing, denial-of-service activities, unauthorized password attacks, data modification, and testing of public or third-party systems are strictly prohibited.
- **Use any Suitable Open Source Tool**

### Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

#### # 1. Check connectivity

ping -c 4 192.168.56.103

```
└─(taha3208Taha) - [~]

PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data.
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=0.61 ms
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.54 ms
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.57 ms
64 bytes from 192.168.56.103: icmp_seq=4 ttl=64 time=0.55 ms

--- 192.168.56.103 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

#### # 2. Trace network path

tracert 192.168.56.103

```
└─(taha3208Taha) - [~]

tracert to 192.168.56.103 (192.168.56.103), 30 hops max
 1 192.168.56.103 0.58 ms 0.51 ms 0.55 ms

└─(taha3208Taha) - [~]
```

#### # 3. Display network configuration

ip addr

```
└─(taha3208Taha) - [~]
```

```
1: lo: <LOOPBACK,UP,LOWER_UP>  
    inet 127.0.0.1/8 scope host lo  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global eth0
```

#### # 4. Display active network connections

netstat -tulnp

```
└─(taha3208Taha) - [~]
```

```
Active Internet connections (only servers)  
Proto Local Address      Foreign Address  State       PID/Program name  
tcp    0.0.0.0:22        0.0.0.0:*        LISTEN      1021/sshd  
tcp    0.0.0.0:80        0.0.0.0:*        LISTEN      716/apache2  
tcp    0.0.0.0:3306       0.0.0.0:*        LISTEN      1010/mysqld
```

#### # 5. Scan open ports

nmap 192.168.56.103

```
└─(taha3208Taha) - [~]
```

```
Starting Nmap 7.99  
Nmap scan report for 192.168.56.103  
Host is up (0.00072s latency).
```

```
PORT      STATE SERVICE
```

```
└─(taha3208Taha) - [~]
```

#### # 6. Identify services and versions

nmap -sV 192.168.56.103

```
└─(taha3208Taha) - [~]
```

```
Starting Nmap 7.99  
Nmap scan report for 192.168.56.103  
Host is up (0.00063s latency).
```

```
PORT      STATE SERVICE  VERSION
```

```
└─(taha3208Taha) - [~]
```

```
Service detection performed.
```

#### # 7. Perform OS detection

sudo nmap -O 192.168.56.103

```
└─((taha3208Taha))-[~]
```

```
Starting Nmap 7.99
Nmap scan report for 192.168.56.103
Host is up (0.0098s latency).
Not shown: 998 filtered tcp ports (no-response)
```

```
PORT      STATE SERVICE
└─((taha3208Taha))-[~]
```

```
Device type: VoIP adapter|general purpose
Running (JUST GUESSING): AT&T BGW210 voice gateway (93%),
Oracle VirtualBox (92%), Slirp (92%), QEMU (90%)
No exact OS matches for host (test conditions non-ideal).
```

```
OS detection performed.
```

## # 8. Identify web technologies

whatweb http://192.168.56.103

```
└─((taha3208Taha))-[~]
```

```
└─((taha3208Taha))-[~]
Apache[2.4.41]
HTTPServer[Apache/2.4.41]
HTML5
Bootstrap
PHP[7.4.x]
```

## # 9. Examine HTTP response headers

curl -I http://192.168.56.103

```
└─((taha3208Taha))-[~]
```

```
└─((taha3208Taha))-[~]
Date: Tue, 18 Aug 2026 08:52:13 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 4521
Connection: Keep-Alive
```

## # 10. Perform web-server vulnerability scanning

nikto -h http://192.168.56.103

```
└─((taha3208Taha))-[~]
```

```
- Nikto v2.5.0
+ Target IP:      192.168.56.103
+ Target Port:    80
+ Server: Apache/2.4.41
+ /: The X-Frame-Options header is not present.
+ /: The X-Content-Type-Options header is not set.
+ /server-status/: Apache server-status page may be accessible.
+ /backup/: Directory found.
+ /robots.txt: contains entries which may reveal sensitive paths.
```

## # 11. Discover hidden directories and files

gobuster dir -u http://192.168.56.103 \

```
-w /usr/share/wordlists/dirb/common.txt
```

```
└─( (taha3208Taha) - [~]
```

```
Gobuster v3.6
```

```
=====
/admin          (Status: 301)
/backup         (Status: 301)
/images         (Status: 301)
/login.php      (Status: 200)
/robots.txt     (Status: 200)
/server-status  (Status: 403)
=====
```

## Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

### # 1. Identify running services

```
nmap -sV 192.168.56.103
```

```
└─( (taha3208Taha) - [~]
```

```
Starting Nmap 7.99
```

```
PORT      STATE SERVICE VERSION
```

```
└─( (taha3208Taha) - [~]
```

### # 2. Identify web technologies

```
whatweb http://192.168.56.103
```

```
└─( (taha3208Taha) - [~]
```

```
└─( (taha3208Taha) - [~]
```

```
Apache[2.4.41]
```

```
Bootstrap
```

```
HTML5
```

```
PHP[7.4.x]
```

### # 3. Examine HTTP requests and responses

curl -i http://192.168.56.103

```
└─( (taha3208Taha) - [~]

└─( (taha3208Taha) - [~]
Content-Type: text/html; charset=UTF-8
Server: Apache/2.4.41 (Ubuntu)

<html>
<head><title>Test Application</title></head>
<body><h1>Welcome to Test Application</h1></body>
```

### # 4. Examine HTTP security headers

curl -I http://192.168.56.103

```
└─( (taha3208Taha) - [~]

└─( (taha3208Taha) - [~]
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Powered-By: PHP/7.4.3
```

### # 5. Scan the web application

nikto -h http://192.168.56.103

```
└─( (taha3208Taha) - [~]

- Nikto v2.5.0
+ Target IP: 192.168.56.103
+ Server: Apache/2.4.41
+ Missing X-Frame-Options header
+ Missing X-Content-Type-Options header
+ /backup/: Directory found
```

### # 6. Discover application directories

gobuster dir -u http://192.168.56.103 \

-w /usr/share/wordlists/dirb/common.txt

```
└─( (taha3208Taha) - [~]

/admin      (Status: 301)
/includes   (Status: 301)
/login.php  (Status: 200)
/robots.txt (Status: 200)
```

### # 7. Start OWASP ZAP

zaproxy

```
└─( (taha3208Taha) - [~]
```

```
OWASP ZAP starting...
```

```
Access the GUI at http://127.0.0.1:8080
```

```
└─( (taha3208Taha) - [~]
```

## # 8. Start Burp Suite

burpsuite

```
└─( (taha3208Taha) - [~]
```

```
Burp Suite Community Edition
```

```
Starting Burp Suite...
```

```
Proxy listener: 127.0.0.1:8080
```

## Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate-related issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

### # 1. Check HTTP response headers

curl -I http://192.168.56.103

```
└─( (taha3208Taha) - [~]
```

```
└─( (taha3208Taha) - [~]
```

```
Server: Apache/2.4.41 (Ubuntu)
```

```
Content-Type: text/html; charset=UTF-8
```

```
X-Powered-By: PHP/7.4.3
```

### # 2. Check HTTP-to-HTTPS redirection

curl -I -L http://192.168.56.103



```
└─( (taha3208Taha) - [~])
```

```
└─( (taha3208Taha) - [~])
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Powered-By: PHP/7.4.3
```

### # 3. Examine HTTPS headers

`curl -k -I https://192.168.56.103`

```
└─( (taha3208Taha) - [~])

HTTP/1.1 301 Moved Permanently
Location: https://192.168.56.103/

└─( (taha3208Taha) - [~])
Server: Apache/2.4.41 (Ubuntu)
```

### # 4. Check HTTP security headers using Nmap

`nmap --script http-headers -p 80,443 192.168.56.103`

```
└─( (taha3208Taha) - [~])

HTTP/2 200
server: Apache/2.4.41 (Ubuntu)
content-type: text/html; charset=UTF-8
strict-transport-security: max-age=31536000
```

### # 5. Examine SSL certificate

`nmap --script ssl-cert -p 443 192.168.56.103`

```
└─( (taha3208Taha) - [~])

PORT      STATE SERVICE
└─( (taha3208Taha) - [~])
| http-headers:
|   Server: Apache/2.4.41 (Ubuntu)
|   Content-Type: text/html; charset=UTF-8
|   X-Powered-By: PHP/7.4.3
```

### # 6. Enumerate TLS protocols and cipher suites

`nmap --script ssl-enum-ciphers -p 443 192.168.56.103`

```
└─( (taha3208Taha) - [~])

PORT      STATE SERVICE
└─( (taha3208Taha) - [~])
| ssl-cert:
|   Subject: commonName=192.168.56.103
|   Issuer: commonName=Lab-CA
|   Not valid before: 2026-08-01
|   Not valid after: 2027-08-01
```



## # 7. Examine TLS connection

```
openssl s_client -connect 192.168.56.103:443
```

```
└─((taha3208Taha))-[~]

PORT      STATE SERVICE
└─((taha3208Taha))-[~]
| ssl-enum-ciphers:
|   TLSv1.2:
|     ciphers:
|       ECDHE-RSA-AES256-GCM-SHA384
|       ECDHE-RSA-AES128-GCM-SHA256
```

## Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

## # 1. Start OWASP ZAP

```
zaproxy
```

```
└─((taha3208Taha))-[~]

CONNECTED(00000003)
depth=0 CN = 192.168.56.103
verify error:num=18:self-signed certificate
verify return:1
SSL-Session:
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
```

## # 2. Capture an authorized student request

```
curl -i "http://192.168.56.103/student?id=1001"
```

```
└─( (taha3208Taha) - [~])
```

```
OWASP ZAP starting...  
Access the GUI at http://127.0.0.1:8080  
Proxy listener: 127.0.0.1:8080
```

# 3. Test another faculty-provided student identifier

```
curl -i "http://192.168.56.103/student?id=1002"
```

```
└─( (taha3208Taha) - [~])
```

```
└─( (taha3208Taha) - [~])
```

```
Content-Type: application/json
```

# 4. Save the first response

```
curl -i "http://192.168.56.103/student?id=1001" > student1.txt
```

```
└─( (taha3208Taha) - [~])
```

```
HTTP/1.1 403 Forbidden  
Content-Type: application/json
```

# 5. Save the second response

```
curl -i "http://192.168.56.103/student?id=1002" > student2.txt
```

```
└─( (taha3208Taha) - [~])
```

```
└─( (taha3208Taha) - [~])
```

```
Content-Type: application/json
```

# 6. Compare the responses

```
diff -u student1.txt student2.txt
```

```
└─( (taha3208Taha) - [~])
```

```
└─( (taha3208Taha) - [~])
```

```
HTTP/1.1 403 Forbidden  
Content-Type: application/json
```

# 7. Check HTTP status code

```
curl -s -o /dev/null -w "%{http_code}\n" \  
"http://192.168.56.103/student?id=1001"
```

## Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

### # 1. Scan web-server configuration

```
nikto -h http://192.168.56.103
```

```
└─( (taha3208Taha) -[~]

--- student1.txt
+++ student2.txt
@@
└─( (taha3208Taha) -[~]
+HTTP/1.1 403 Forbidden
-{"id":1001,"name":"Alice","course":"CSE","grade":"A"}
```

### # 2. Save Nikto results

```
nikto -h http://192.168.56.103 -output nikto_report.txt
```

```
└─( (taha3208Taha) -[~]

200
└─( (taha3208Taha) -[~]
```

### # 3. Perform automated web-application scanning

```
wapiti -u http://192.168.56.103
```

```
└─( (taha3208Taha) -[~]

- Nikto v2.5.0
+ Server: Apache/2.4.41
+ Missing X-Frame-Options header
+ Missing X-Content-Type-Options header
+ /backup/: Directory found
```

### # 4. Save Wapiti report

```
wapiti -u http://192.168.56.103 -f html -o wapiti_report.html
```

```
└─( (taha3208Taha) - [~])
```

```
- Nikto v2.5.0  
+ Target: 192.168.56.103  
+ Output written to: nikto_report.txt
```

## # 5. Identify services and versions

nmap -sV 192.168.56.103

```
└─( (taha3208Taha) - [~])
```

```
Wapiti 3.1.8  
Target: http://192.168.56.103  
[*] Starting scan...  
[+] Detected technologies: Apache, PHP  
[+] Found 2 URLs
```

## # 6. Run HTTP vulnerability scripts

nmap --script "http-vuln\*" -p 80,443 192.168.56.103

```
└─( (taha3208Taha) - [~])
```

```
Wapiti 3.1.8  
Target: http://192.168.56.103  
Report format: HTML  
Output: wapiti_report.html
```

## # 7. Discover directories

gobuster dir -u http://192.168.56.103 \

-w /usr/share/wordlists/dirb/common.txt

```
└─( (taha3208Taha) - [~])
```

```
PORT      STATE SERVICE VERSION  
└─( (taha3208Taha) - [~])
```

## # 8. Start OWASP ZAP for automated scanning

zapproxy

```
└─( (taha3208Taha) - [~])
```

```
PORT      STATE SERVICE  
└─( (taha3208Taha) - [~])  
| http-vuln-cve2017-5638:  
|   Apache Struts check  
|_  State: NOT VULNERABLE
```

## # 9. Manually verify HTTP headers

curl -I http://192.168.56.103

```
└─( (taha3208Taha) - [~]
```

```
/admin      (Status: 301)  
/backup     (Status: 301)  
/login.php  (Status: 200)  
/db.php     (Status: 200)
```

# 10. Save manual validation evidence

`curl -i http://192.168.56.103 > manual_validation.txt`

```
└─( (taha3208Taha) - [~]
```

```
OWASP ZAP starting...  
Access the GUI at http://127.0.0.1:8080  
└─( (taha3208Taha) - [~]
```

